

CÓDIGO FÁCIL

MANUAL TÉCNICO DEL SISTEMA

Implementación de un sistema de gestión de jornada laboral en la empresa Código
Fácil

Desarrollador:

Joshua Díaz Espinoza

Fecha de elaboración:

10 de abril de 2026

Versión del documento:

1.0

Índice

Tabla de ilustraciones.....	8
Introducción	9
Descripción general del Sistema	9
Tecnologías utilizadas	10
1. Frontend	10
2. Backend.....	11
3. Base de Datos.....	11
4. Autenticación y Seguridad.....	12
5. Herramientas de Desarrollo	12
6. Despliegue.....	12
Instalación del sistema	13
2. Requisitos del sistema.....	13
3. Instalación de Node.js	13
Paso 1: Instalación de Node.js	13
Paso 2: Seleccionar la versión correcta.....	14
Paso 3: Descargar el instalador	14
Paso 4: Ejecutar la instalación	15
Paso 5: Verificar la instalación.....	15
Paso 6: Ejecución del sistema	16
Organización general del proyecto	16
1.1. Estructura de carpetas principales.....	17
1.2. Arquitectura del sistema.....	19
Diagramas de flujos	20
Diagrama flujo general	20
Diagrama caso de secuencia Crear cuenta:	22
Diagrama de secuencia Crear proyecto:	23
Diagrama de secuencia Unirse proyecto:.....	24
Diagrama de actividades Buscar proyecto:	25
Diagrama de actividades registro jornada:.....	26
Diagrama de clase codigofacildb:	27
Backend.....	27
Base de datos.....	27

1. Tabla: usuarios	27
Función de la tabla	27
Script de creación.....	28
2. Tabla: proyectos	29
Función de la tabla	29
Estructura de la tabla.....	29
Script de creación.....	30
3. Tabla: tareas	31
Estructura de la tabla.....	31
Script de creación.....	32
Estructura de la tabla.....	34
Script de creación.....	34
5. Tabla: tarea_historial.....	35
Función de la tabla	35
Estructura de la tabla.....	35
Script de creación.....	36
Función de la tabla	36
Estructura de la tabla.....	37
Script de creación.....	37
7. Tabla: supervisor_usuarios.....	38
Función de la tabla	38
Script de creación.....	39
8. Tabla: registro_jornada	39
Función de la tabla	39
Estructura de la tabla.....	39
Script de creación.....	40
9. Tabla: registro_horas.....	40
Función de la tabla	40
Estructura de la tabla.....	41
Script de creación.....	41
10. Tabla: proyecto_usuarios	42
Función de la tabla	42

Estructura de la tabla.....	42
Script de creación.....	42
11. Tabla: proyecto_solicitudes	43
Función de la tabla	43
Estructura de la tabla.....	43
Script de creación.....	44
12. Tabla: sesiones_trabajo	45
Función de la tabla	45
Estructura de la tabla.....	45
Script de creación.....	45
Configuración del Entorno (.env.local).....	46
Script:	46
Descripción de variables	46
Conexión a la base de datos	47
(lib/database.ts)	47
Función del archivo.....	47
Código.....	47
Explicación técnica	47
Servicio de envío de correos	48
(lib/email.ts)	48
Función del archivo.....	48
Código.....	48
Explicación técnica	48
Gestión de países.....	48
(lib/countries.ts)	48
Función del archivo.....	48
Código.....	49
Funcionalidades principales:.....	51
Explicación técnica	52
Autenticación	52
(lib/auth.ts)	52
Función del archivo.....	52

Código.....	53
Funciones principales.....	56
Explicación técnica	58
1. Endpoints de autenticación	58
1.1. Inicio de sesión	58
1.2. Cierre de sesión.....	58
1.3. Usuario autenticado	58
1.4. Recuperación de contraseña.....	59
1.5. Verificación de código	59
1.6. Restablecimiento de contraseña.....	59
1.7. Registro de usuario	60
2. Endpoints del módulo de jornada laboral	60
2.1. Consulta de registros de jornada	60
2.2. Registro de jornada	61
2.3. Consulta de usuarios de jornada	61
2.4. Asignación de usuarios a jornada	61
2.5. Actualización de un registro de jornada.....	62
2.6. Eliminación de un registro de jornada	62
3. Endpoints del módulo de proyectos.....	63
3.1. Consulta general de proyectos	63
3.2. Creación de proyecto	63
3.3. Consulta de proyecto específico	63
4. Endpoints avanzados del módulo de proyectos	64
4.1. Consulta de configuración del proyecto	64
4.2. Actualización de configuración del proyecto	64
4.3. Eliminación de proyecto	65
4.4. Consulta de miembros del proyecto.....	65
4.5. Agregar miembro al proyecto	65
4.6. Actualizar rol de miembro	66
4.7. Eliminar miembro del proyecto	66
4.8. Vista previa del proyecto	67
5. Endpoints de acceso y solicitudes de proyectos	67

5.1.	Unirse a un proyecto público	67
5.2.	Crear solicitud de acceso a proyecto.....	68
5.3.	Consultar solicitudes pendientes.....	68
5.4.	Procesar solicitud de acceso.....	69
6.	Endpoints del módulo de tareas	69
6.1.	Consulta de tareas del proyecto	69
6.2.	Creación de tarea.....	70
6.3.	Aprobación de tarea	70
6.4.	Consulta de asignados de una tarea	71
6.5.	Cancelación de selección de tarea.....	71
6.6.	Comenzar tarea (iniciar cronómetro).....	72
6.7.	Enviar tarea a revisión (completar).....	72
6.8.	Actualizar estado de tarea (control total).....	73
6.9.	Informes de tarea (avance / final)	73
6.10.	Rechazar tarea.....	74
6.11.	Seleccionar tarea	75
	Estructura del Backend	75
	Estructura general	76
	Esquema backend.....	76
	Explicacion esquema:	77
	Componentes principales	77
	Flujo del Backend	78
	Lógica del sistema de tareas	78
	Reglas principales:	78
	Control de tiempo (registro_horas).....	79
	Estados del registro:.....	79
	Funcionalidad:	79
	Seguridad del Backend	80
	1. Autenticación	80
	2. Autorización.....	80
	3. Validaciones	80
	4. Protección de rutas	81

Manejo de errores	81
Buenas prácticas implementadas	81
Conclusión técnica del backend.....	82

Tabla de ilustraciones

Diagrama 1: Diagrama de flujo 1.....	20
Diagrama 2: Diagrama de flujo 2.....	21
Diagrama 3:Diagrama de Actividades 1	22
Diagrama 4; :Diagrama de Actividades 2	23
Diagrama 5: :Diagrama de Actividades 3	24
Diagrama 6: :Diagrama de Actividades 4	25
Diagrama 7: :Diagrama de Actividades 5	26
Diagrama 8: Diagrama de clases DB.....	27

Introducción

El presente manual técnico tiene como propósito documentar la estructura, funcionamiento e implementación del sistema de gestión de proyectos, tareas y control de jornada laboral desarrollado como solución tecnológica para optimizar procesos organizacionales.

Este documento está dirigido a personal técnico, desarrolladores y administradores del sistema, proporcionando la información necesaria para su instalación, configuración, mantenimiento y comprensión de su arquitectura interna.

El sistema permite gestionar de manera eficiente proyectos, asignar tareas a los colaboradores, controlar el progreso de las actividades y registrar las horas laborales, facilitando así la toma de decisiones y el seguimiento del desempeño dentro de la organización.

Descripción general del Sistema

El sistema desarrollado es una aplicación web orientada a la gestión integral de proyectos y tareas, complementada con un módulo de control de jornada laboral. Su objetivo principal es mejorar la organización, supervisión y control de actividades dentro de una empresa o equipo de trabajo.

La aplicación permite a los usuarios crear y administrar proyectos, asignar tareas a diferentes colaboradores, establecer prioridades, dar seguimiento al estado de las actividades y generar reportes relacionados con el avance del trabajo. Además, incorpora un sistema de roles y permisos que garantiza un acceso controlado a las distintas funcionalidades.

Desde el punto de vista técnico, el sistema ha sido desarrollado utilizando tecnologías modernas como Next.js y React para el frontend, junto con API Routes para la lógica del backend. La base de datos se gestiona mediante Turso (basado en SQLite), lo que permite un manejo eficiente y escalable de la información.

Asimismo, el sistema incluye funcionalidades como:

- Gestión de usuarios y roles (administrador, jefe, colaborador).
- Creación, asignación y seguimiento de tareas.
- Flujo de estados de tareas (pendiente, en progreso, revisión y completado).
- Registro y control de jornada laboral.
- Generación de reportes para análisis y control.

En conjunto, este sistema proporciona una solución integral que contribuye a la eficiencia operativa, la organización del trabajo y el control del desempeño dentro de la empresa.

Tecnologías utilizadas

El sistema fue desarrollado utilizando un conjunto de tecnologías modernas orientadas al desarrollo web, las cuales permiten garantizar un alto rendimiento, escalabilidad y facilidad de mantenimiento.

A continuación, se describen las principales tecnologías empleadas:

1. Frontend

- **Next.js**

Framework basado en React utilizado para el desarrollo de la aplicación web, permitiendo la creación de interfaces dinámicas y el manejo de rutas mediante el App Router.

- **React**

Biblioteca utilizada para la construcción de componentes reutilizables y la gestión del estado de la interfaz de usuario.

- **Tailwind CSS**

Framework de estilos que facilita el diseño responsivo y la personalización de la interfaz mediante clases utilitarias.

2. Backend

- **API Routes de Next.js**

Utilizadas para la implementación de la lógica del servidor, permitiendo manejar solicitudes HTTP (GET, POST, PATCH, DELETE) directamente dentro del mismo proyecto.

- **Node.js**

Entorno de ejecución que permite ejecutar código JavaScript en el servidor.

3. Base de Datos

- **Turso**

Base de datos distribuida basada en SQLite, utilizada para el almacenamiento de la información del sistema de manera eficiente.

- **SQLite**

Motor de base de datos relacional ligero, empleado como base para la gestión de datos.

4. Autenticación y Seguridad

- **Cookies de sesión**

Utilizadas para mantener la autenticación del usuario dentro del sistema.

- **bcrypt**

Librería utilizada para el cifrado de contraseñas, garantizando la seguridad de la información del usuario.

5. Herramientas de Desarrollo

- **Visual Studio Code**

Editor de código fuente utilizado para el desarrollo del sistema.

- **Git**

Sistema de control de versiones utilizado para el manejo del código fuente.

- **GitHub**

Plataforma utilizada para el almacenamiento y gestión del repositorio del proyecto.

6. Despliegue

- **Vercel**

Plataforma utilizada para el despliegue y publicación de la aplicación web.

Instalación del sistema

1. Acceso al repositorio

El código fuente del sistema se encuentra disponible en el siguiente repositorio:

GitHub:

<https://github.com/JoshuaDE506/control-horas>

Desde este repositorio, el usuario puede clonar o descargar el proyecto para su ejecución local.

2. Requisitos del sistema

Para la correcta instalación y ejecución del sistema, se requieren los siguientes componentes:

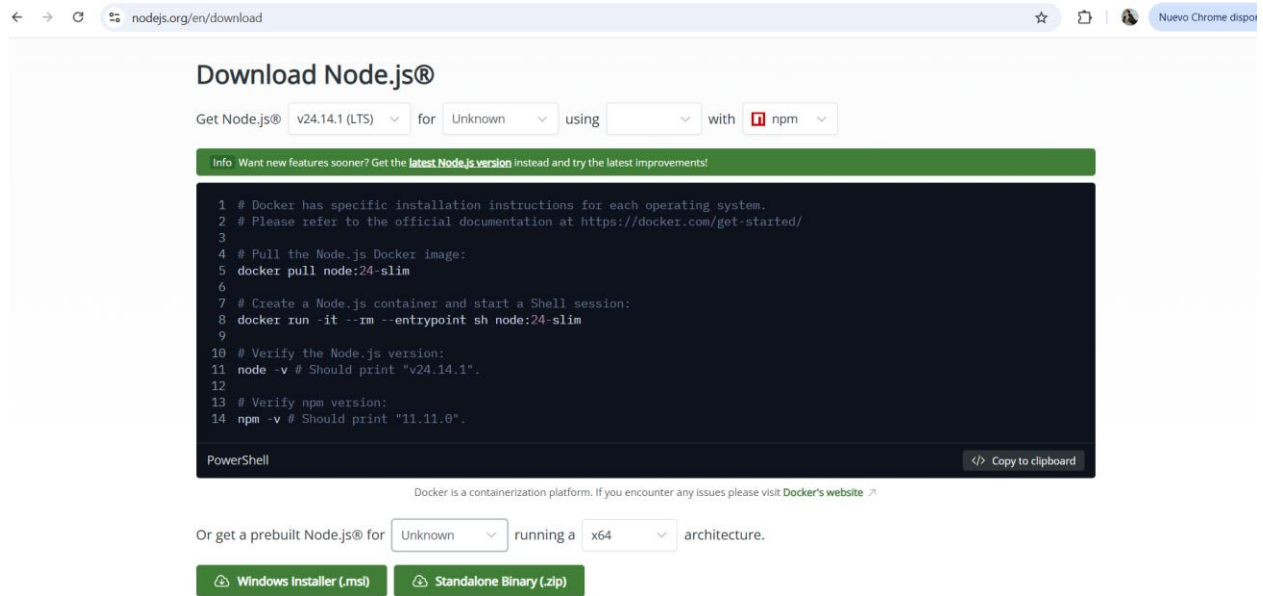
- Node.js (versión 18 o superior)
- npm (gestor de paquetes)
- Navegador web moderno
- Cuenta activa en Turso

3. Instalación de Node.js

Paso 1: Instalación de Node.js

Descargar e instalar Node.js desde su sitio oficial.

URL: <https://nodejs.org/en/download>



Instalacion: 1 Pagina Node.Js

Paso 2: Seleccionar la versión correcta

En la página se muestran distintas versiones disponibles.

Se recomienda seleccionar la versión:

- **LTS (Long Term Support):**

Es la versión más estable y recomendada para entornos empresariales

Paso 3: Descargar el instalador

El usuario debe hacer clic en el botón:

- **Windows Installer (.msi)** (para sistemas Windows)

Esto descargará el instalador oficial de Node.js.

Paso 4: Ejecutar la instalación

Una vez descargado el archivo:

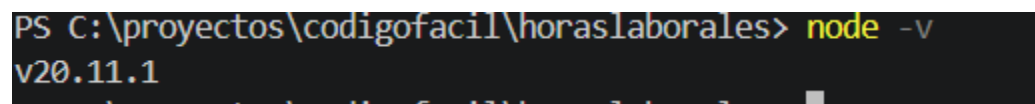
1. Abrir el archivo .msi
2. Seguir el asistente de instalación
3. Aceptar términos y condiciones
4. Dejar las opciones por defecto
5. Finalizar la instalación

Paso 5: Verificar la instalación

Abrir la aplicación Terminal y ejecutar:

node -v

Debe visualizarse así:



```
PS C:\proyectos\codigofacil\horaslaborales> node -v
v20.11.1
```

Comando 1: Version de node

Y después ejecutar:

npm -v

Debe visualizarse de la siguiente forma:

```
PS C:\proyectos\codigofacil\horaslaborales> npm -v
10.2.4
```

Comando 2: version npm

Paso 6: Ejecución del sistema

Una vez verificada la instalación de Node.js y npm, se debe proceder a ejecutar el sistema de forma local. Para ello, el usuario debe abrir la terminal, ubicarse en la carpeta donde se descargó o clonó el proyecto y ejecutar los siguientes comandos:

npm install

npm run dev

El comando `npm install` se encarga de instalar todas las dependencias necesarias del sistema, mientras que `npm run dev` inicia el servidor en modo de desarrollo.

Una vez ejecutado el sistema, este estará disponible en la siguiente dirección:

<http://localhost:3000>

Desde esta dirección, el usuario podrá acceder a la aplicación web, iniciar sesión y utilizar todas sus funcionalidades.

Organización general del proyecto

El sistema está estructurado siguiendo la arquitectura de Next.js, utilizando el App Router para la gestión de rutas y vistas.

La organización del proyecto se basa en una estructura modular que facilita el mantenimiento, escalabilidad y reutilización del código.

1.1.Estructura de carpetas principales

A continuación, se describen las carpetas más importantes del proyecto:

1. /app

Contiene las vistas principales del sistema y la configuración de rutas.

- Maneja la navegación entre páginas
- Define las interfaces del usuario
- Incluye páginas como dashboard, proyectos y tareas

2. /app/api

- Contiene los endpoints del backend.
- Maneja las solicitudes HTTP (GET, POST, PATCH, DELETE)
- Controla la lógica del sistema
- Se conecta con la base de datos

Ejemplo:

/api/auth → autenticación

/api/proyectos → gestión de proyectos

/api/tareas → gestión de tareas

3. /components

Contiene componentes reutilizables del sistema como:

- Modales
- Formularios
- Botones
- Tablas

Esto permite mantener un código limpio y reutilizable.

4. /lib

Contiene funciones auxiliares y configuración del sistema.

- Conexión a la base de datos
- Funciones de autenticación
- Utilidades generales

5. /public

- Almacena archivos estáticos como imágenes o íconos.

1.2. Arquitectura del sistema

El sistema sigue una arquitectura basada en:

- Frontend (cliente) → Interfaz de usuario
- Backend (API Routes) → Lógica del sistema
- Base de datos → Almacenamiento de información

El flujo de funcionamiento es el siguiente:

1. El usuario interactúa con la interfaz
2. Se realiza una solicitud a la API
3. La API procesa la información
4. Se consulta o actualiza la base de datos
5. Se devuelve una respuesta al usuario

Diagramas de flujos

Diagrama flujo general

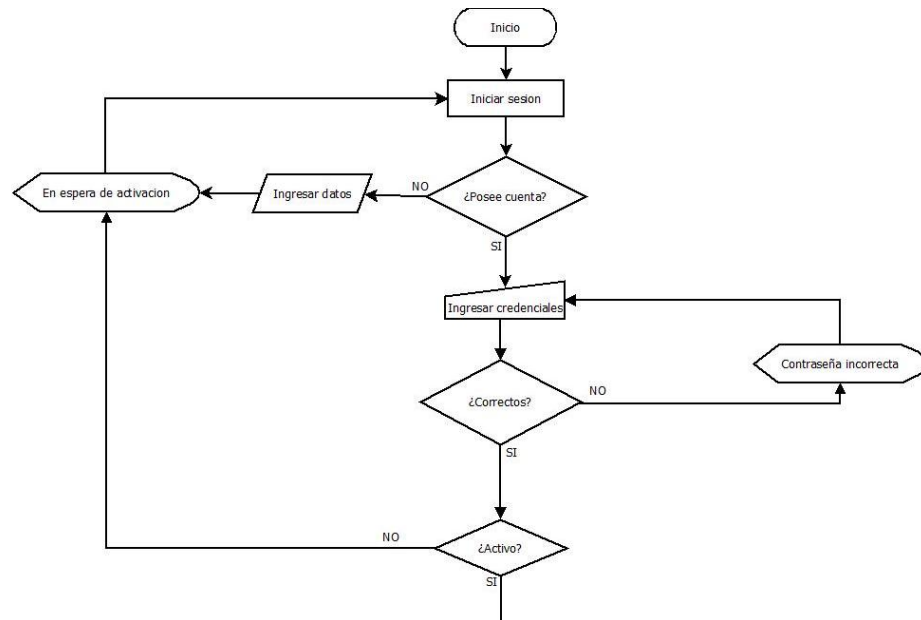


Diagrama 1: Diagrama de flujo 1

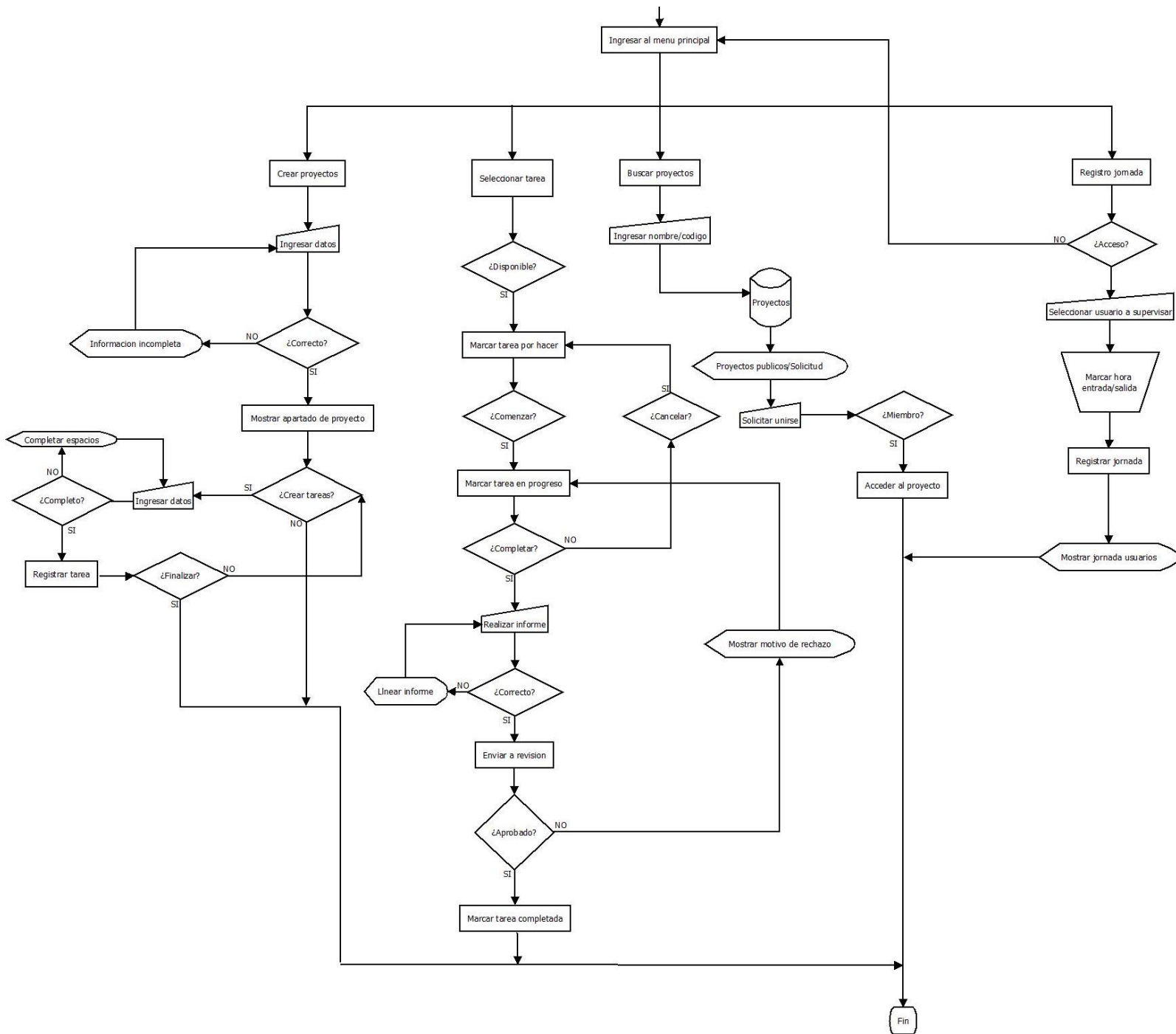


Diagrama 2: Diagrama de flujo 2

Diagrama caso de actividades Crear cuenta:

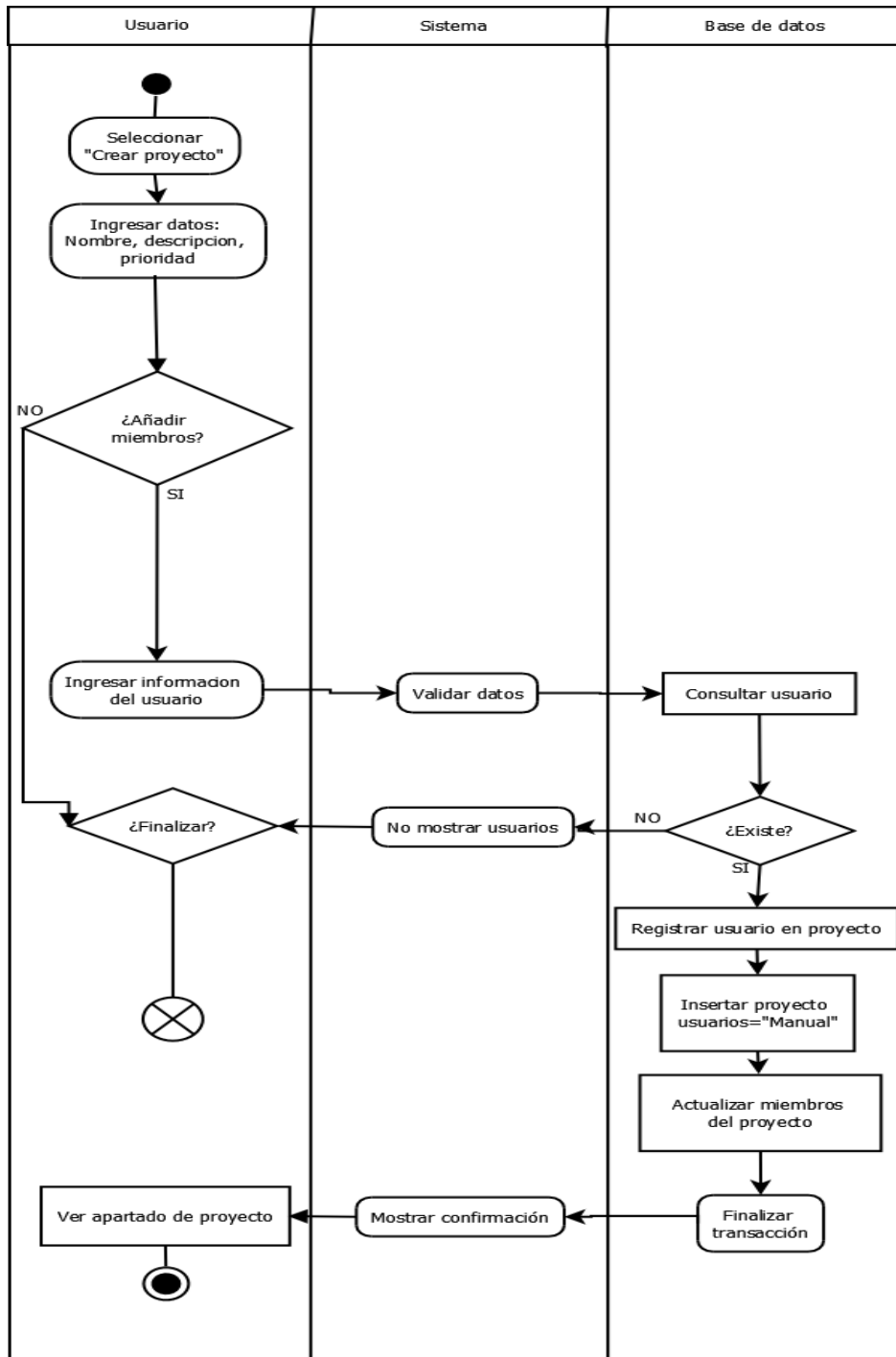


Diagrama 3: Diagrama de Actividades 1

Diagrama de actividades Crear proyecto:

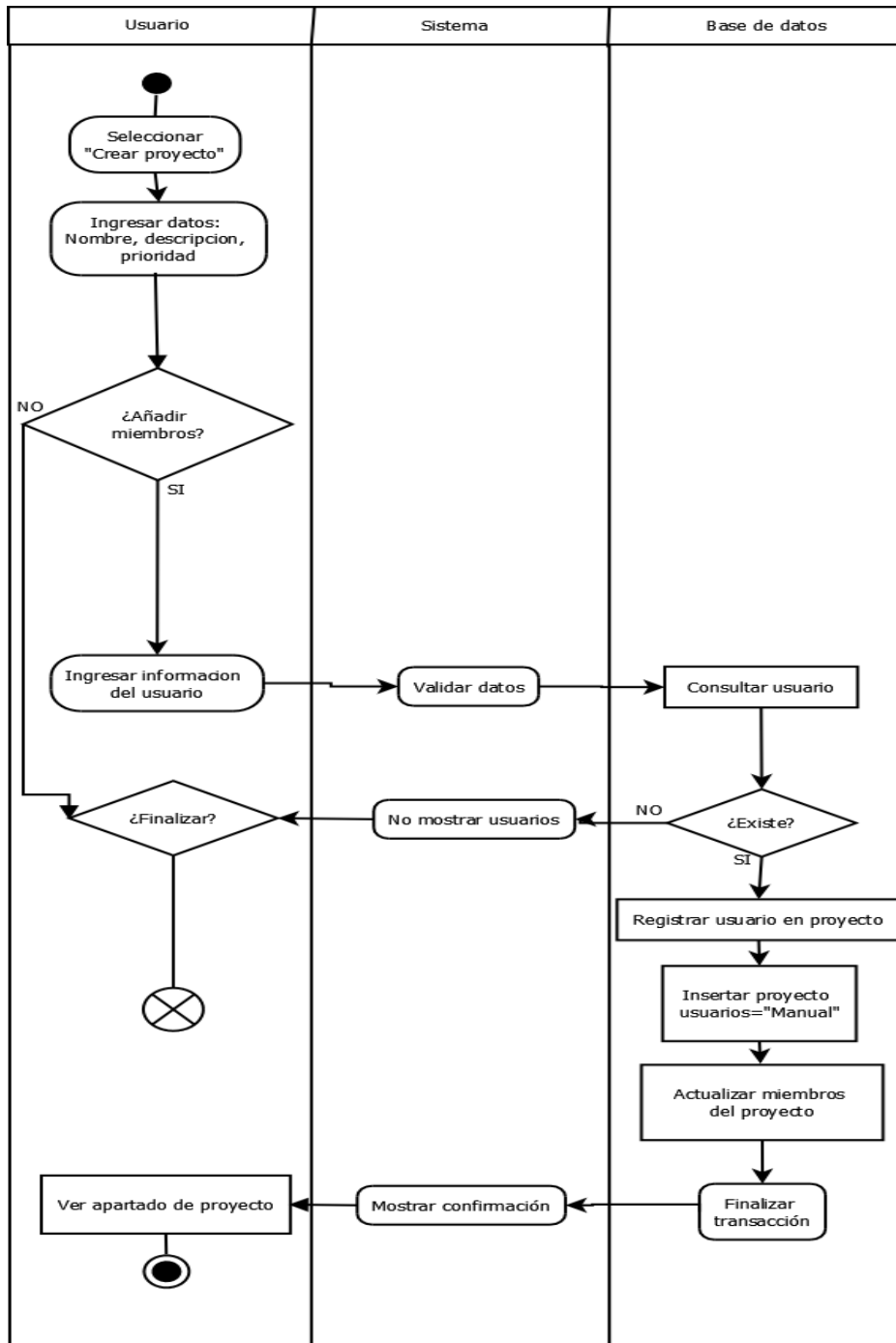


Diagrama 4; :Diagrama de Actividades 2

Diagrama de actividades Unirse proyecto:

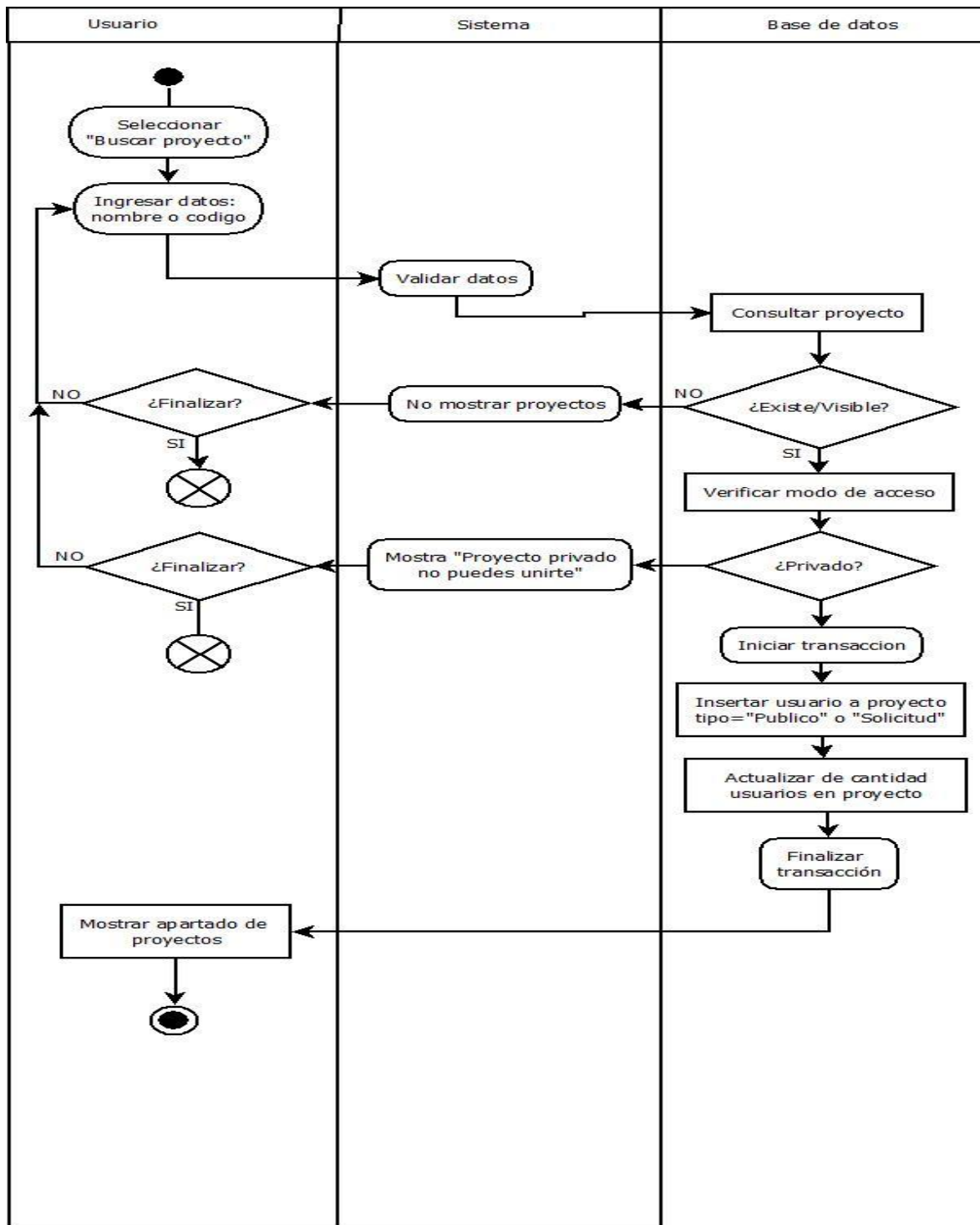


Diagrama 5: :Diagrama de Actividades 3

Diagrama de actividades Buscar proyecto:

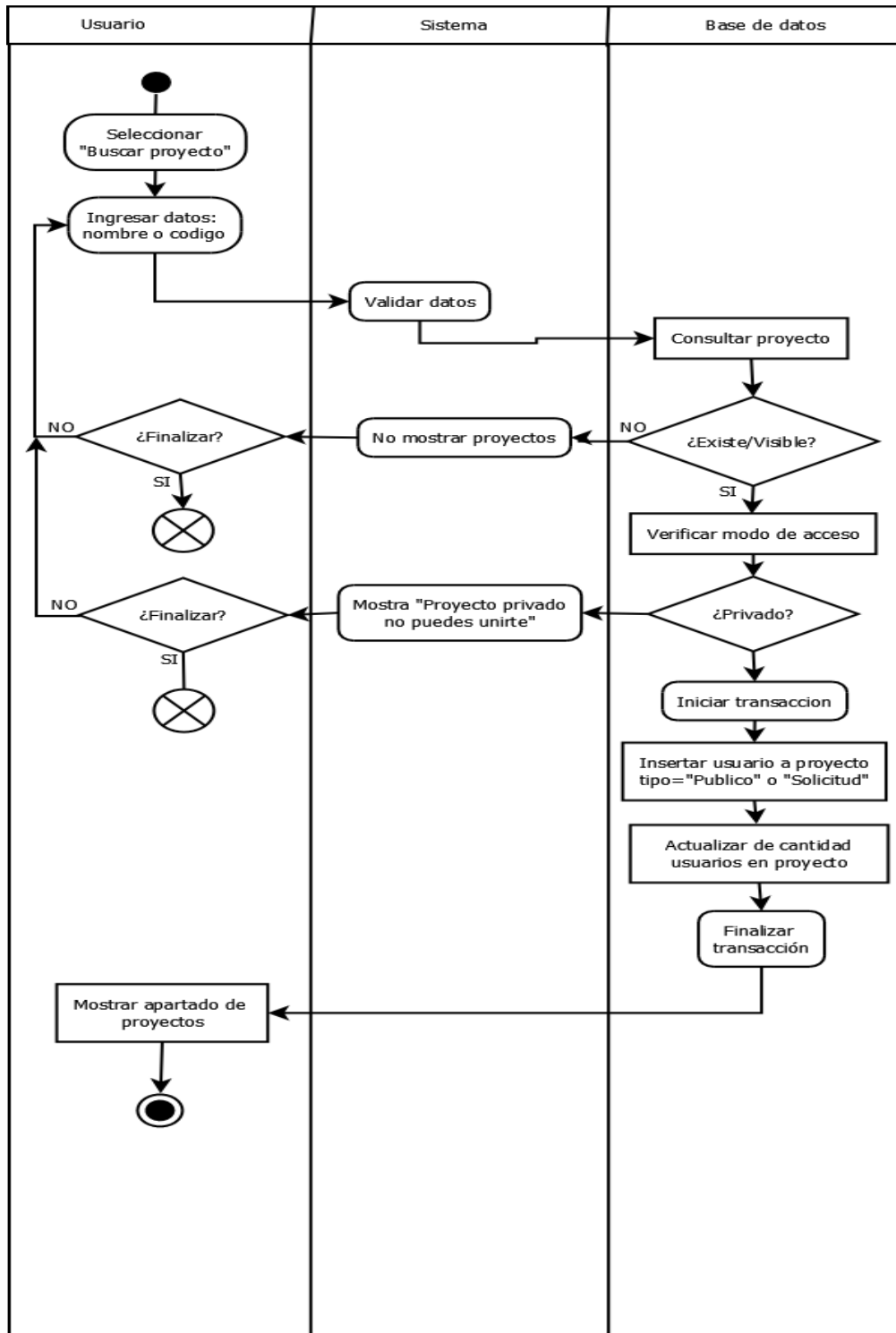


Diagrama 6: :Diagrama de Actividades 4

Diagrama de actividades registro jornada:

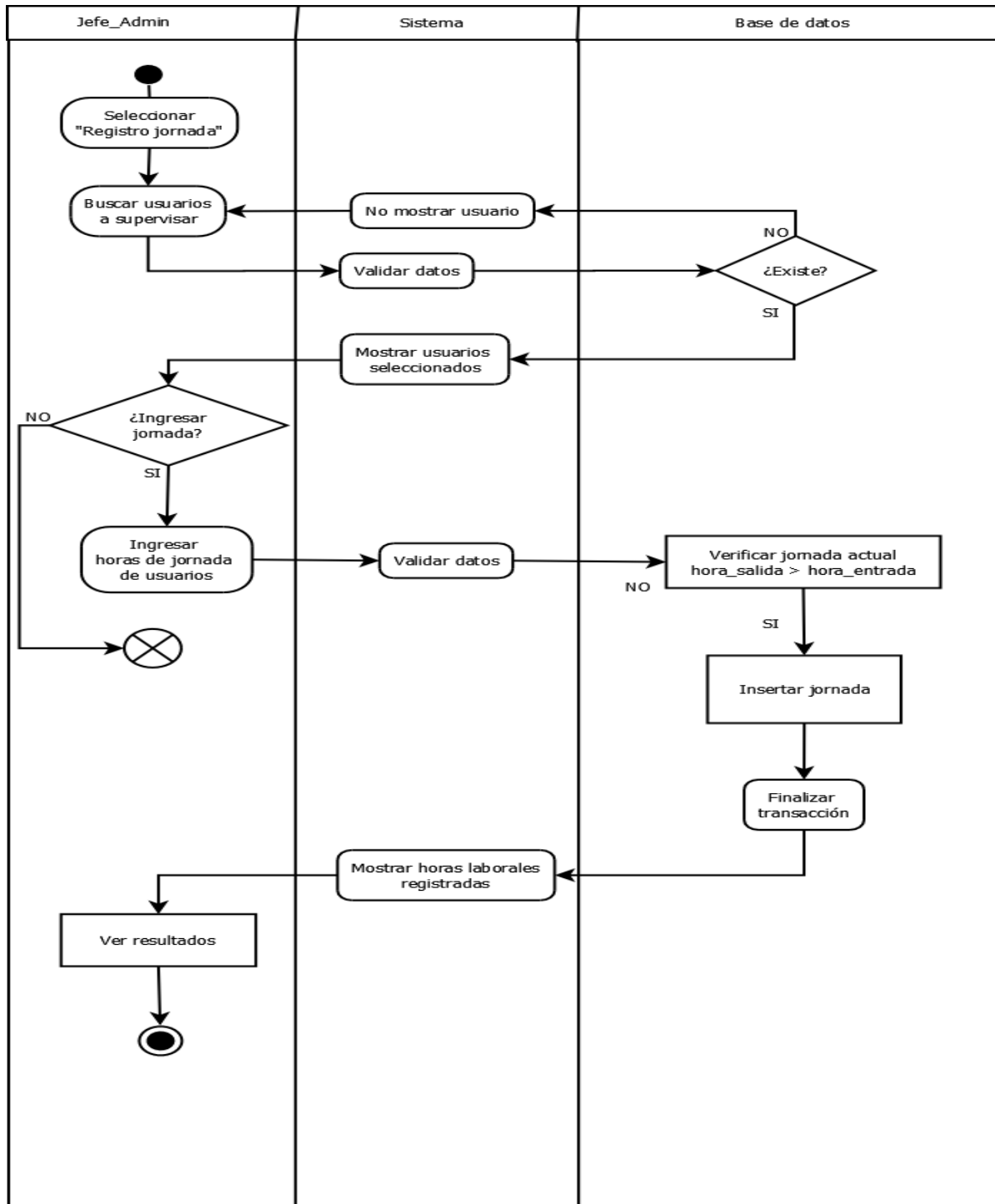


Diagrama 7: :Diagrama de Actividades 5

Diagrama de clase codigofacilddb:

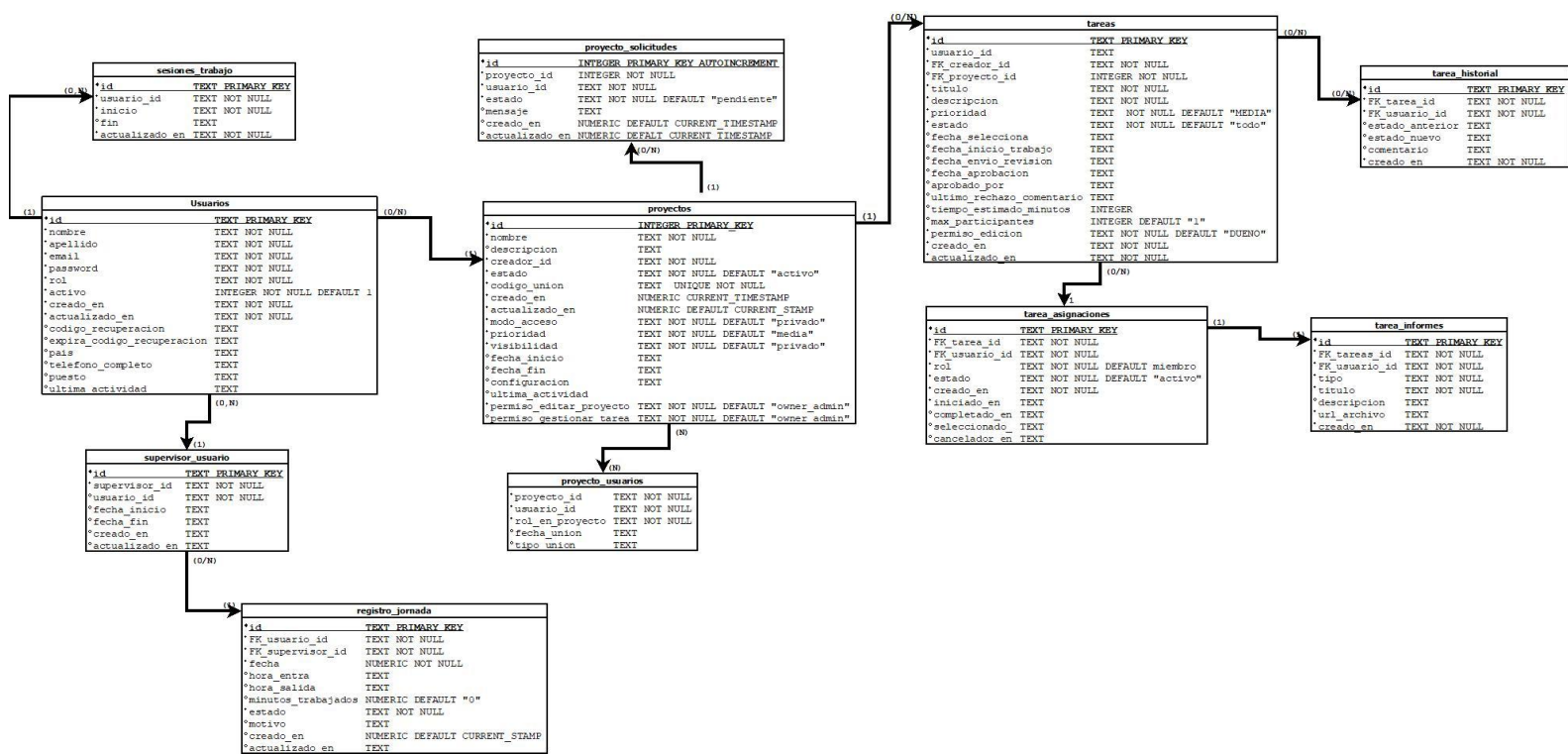


Diagrama 8: Diagrama de clases DB

Backend

Base de datos

1. Tabla: usuarios

Función de la tabla:

La tabla usuarios almacena la información general de las personas que utilizan el sistema. En ella se registran los datos personales, credenciales de acceso, rol dentro de la plataforma, estado de activación y otros campos complementarios como país, teléfono, puesto y última actividad.

- Estructura de la tabla
- id: identificador único del usuario.
- nombre: nombre del usuario.
- apellido: apellido del usuario.
- email: correo electrónico único.
- password: contraseña cifrada del usuario.
- rol: rol asignado dentro del sistema.
- activo: indica si el usuario está habilitado.
- creado_en: fecha de creación del registro.
- actualizado_en: fecha de última actualización.
- codigo_recuperacion: código temporal para recuperación de contraseña.
- expira_codigo_recuperacion: fecha de expiración del código.
- pais: país del usuario.
- telefono_completo: número telefónico completo.
- puesto: cargo o puesto laboral.
- ultima_actividad: última actividad registrada en el sistema.

Script de creación

```
CREATE TABLE usuarios (  
    id TEXT PRIMARY KEY,  
    nombre TEXT NOT NULL,  
    apellido TEXT NOT NULL,  
    email TEXT UNIQUE NOT NULL,  
    password TEXT NOT NULL,  
    rol TEXT NOT NULL,
```

```
activo INTEGER DEFAULT 1,  
creado_en TEXT NOT NULL,  
actualizado_en TEXT NOT NULL,  
codigo_recuperacion TEXT,  
expira_codigo_recuperacion TEXT,  
pais TEXT,  
telefono_completo TEXT,  
puesto TEXT,  
ultima_actividad TEXT  
);
```

2. Tabla: proyectos

Función de la tabla

La tabla proyectos almacena la información principal de cada proyecto creado en el sistema. Incluye datos descriptivos, configuraciones de acceso, prioridad, visibilidad, fechas y permisos de edición y gestión de tareas.

Estructura de la tabla

- id: identificador único del proyecto.
- nombre: nombre del proyecto.
- descripcion: descripción general.
- creador_id: usuario creador del proyecto.
- estado: estado actual del proyecto.
- codigo_union: código único para unirse al proyecto.
- creado_en: fecha de creación.
- actualizado_en: fecha de actualización.

- modo_acceso: define si el acceso es privado, público o por solicitud.
- prioridad: nivel de prioridad del proyecto.
- visibilidad: visibilidad del proyecto.
- fecha_inicio: fecha de inicio planificada.
- fecha_fin: fecha de finalización planificada.
- configuracion: configuraciones adicionales del proyecto.
- ultima_actividad: último movimiento registrado.
- permiso_editar_proyecto: define quién puede editar el proyecto.
- permiso_gestionar_tareas: define quién puede administrar tareas.

Script de creación

```
CREATE TABLE proyectos (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nombre TEXT NOT NULL,
  descripcion TEXT,
  creador_id TEXT NOT NULL,
  estado TEXT NOT NULL DEFAULT 'activo',
  codigo_union TEXT UNIQUE NOT NULL,
  creado_en NUMERIC DEFAULT CURRENT_TIMESTAMP,
  actualizado_en NUMERIC DEFAULT CURRENT_TIMESTAMP,
  modo_acceso TEXT NOT NULL DEFAULT 'privado',
  prioridad TEXT NOT NULL DEFAULT 'media',
  visibilidad TEXT NOT NULL DEFAULT 'privado',
  fecha_inicio TEXT,
  fecha_fin TEXT,
  configuracion TEXT,
```

```

ultima_actividad TEXT,
permiso_editar_proyecto TEXT NOT NULL DEFAULT 'owner_admin',
permiso_gestionar_tareas TEXT NOT NULL DEFAULT 'owner_admin',

CONSTRAINT proyectos_estado_check CHECK (estado IN ('activo', 'pausado', 'completado',
'cancelado')),

CONSTRAINT proyectos_modulo_acceso_check CHECK (modulo_acceso IN ('privado',
'publico', 'solicitud')),

CONSTRAINT proyectos_prioridad_check CHECK (prioridad IN ('baja', 'media', 'alta',
'critica')),

CONSTRAINT proyectos_visibilidad_check CHECK (visibilidad IN ('privado', 'publico')),

CONSTRAINT proyectos_permiso_editar_check CHECK (permiso_editar_proyecto IN
('owner', 'owner_admin', 'all_members')),

CONSTRAINT proyectos_permiso_tareas_check CHECK (permiso_gestionar_tareas IN
('owner', 'owner_admin', 'all_members')),

CONSTRAINT proyectos_creador_id_fk FOREIGN KEY (creador_id) REFERENCES
usuarios(id)
);

```

3. Tabla: tareas

Función de la tabla

La tabla tareas almacena las actividades que forman parte de los proyectos. Permite registrar su creador, asignación, estado, prioridad, tiempos estimados, fechas importantes del flujo de trabajo y datos de aprobación o rechazo.

Estructura de la tabla

- id: identificador único de la tarea.
- usuario_id: usuario asignado directamente.
- creador_id: usuario creador de la tarea.

- proyecto_id: proyecto al que pertenece.
- titulo: nombre o título de la tarea.
- descripcion: detalle de la tarea.
- prioridad: prioridad de la tarea.
- estado: estado actual de la tarea.
- fecha_seleccionada: fecha de selección de la tarea.
- fecha_inicio_trabajo: fecha de inicio de trabajo.
- fecha_envio_revision: fecha de envío a revisión.
- fecha_aprobacion: fecha de aprobación final.
- aprobado_por: usuario que aprobó la tarea.
- ultimo_rechazo_comentario: comentario asociado al último rechazo.
- tiempo_estimado_minutos: tiempo estimado de ejecución.
- max_participantes: cantidad máxima de participantes.
- permiso_edicion: nivel de permiso para editar la tarea.
- creado_en: fecha de creación.
- actualizado_en: fecha de actualización.

Script de creación

```
CREATE TABLE tareas (  
    id TEXT PRIMARY KEY,  
    usuario_id TEXT,  
    creador_id TEXT NOT NULL,  
    proyecto_id INTEGER,  
    titulo TEXT NOT NULL,  
    descripcion TEXT,
```

```

prioridad TEXT NOT NULL DEFAULT 'media',
estado TEXT NOT NULL DEFAULT 'todo',
fecha_seleccionada TEXT,
fecha_inicio_trabajo TEXT,
fecha_envio_revision TEXT,
fecha_aprobacion TEXT,
aprobado_por TEXT,
ultimo_rechazo_comentario TEXT,
tiempo_estimado_minutos INTEGER,
max_participantes INTEGER DEFAULT 1,
permiso_edicion TEXT NOT NULL DEFAULT 'dueno',
creado_en TEXT NOT NULL,
actualizado_en TEXT NOT NULL,

CONSTRAINT tareas_prioridad_check CHECK (prioridad IN ('baja', 'media', 'alta', 'critica')),
CONSTRAINT tareas_estado_check CHECK (estado IN ('todo', 'in-progress', 'review',
'completed')),
CONSTRAINT tareas_max_participantes_check CHECK (max_participantes >= 1),
CONSTRAINT tareas_usuario_id_fk FOREIGN KEY (usuario_id) REFERENCES
usuarios(id),
CONSTRAINT tareas_creador_id_fk FOREIGN KEY (creador_id) REFERENCES
usuarios(id),
CONSTRAINT tareas_proyecto_id_fk FOREIGN KEY (proyecto_id) REFERENCES
proyectos(id),
CONSTRAINT tareas_aprobado_por_fk FOREIGN KEY (aprobado_por) REFERENCES
usuarios(id)
);

CREATE INDEX idx_tareas_estado ON tareas (estado);
CREATE INDEX idx_tareas_proyecto_id ON tareas (proyecto_id);

```

```
CREATE INDEX idx_tareas_usuario_id ON tareas (usuario_id);  
CREATE INDEX idx_tareas_creador_id ON tareas (creador_id);  
CREATE INDEX idx_tareas_creado_en ON tareas (creado_en);
```

4. Tabla: tarea_informes

Función de la tabla

La tabla tarea_informes almacena los informes asociados a las tareas, ya sea de avance o finales. Permite documentar el progreso y entregar evidencia relacionada con el desarrollo de una actividad.

Estructura de la tabla

- id: identificador único del informe.
- tarea_id: tarea a la que pertenece el informe.
- usuario_id: usuario que genera el informe.
- tipo: tipo de informe.
- titulo: título del informe.
- descripcion: descripción o contenido del informe.
- url_archivo: enlace a archivo adjunto.
- creado_en: fecha de creación.

Script de creación

```
CREATE TABLE tarea_informes (  
    id TEXT PRIMARY KEY,  
    tarea_id TEXT NOT NULL,  
    usuario_id TEXT NOT NULL,  
    tipo TEXT NOT NULL,  
    titulo TEXT,
```

```

descripcion TEXT,
url_archivo TEXT,
creado_en TEXT NOT NULL,

CONSTRAINT tarea_informes_tipo_check CHECK (tipo IN ('avance', 'final')),
CONSTRAINT tarea_informes_usuario_id_fk FOREIGN KEY (usuario_id) REFERENCES
usuarios(id),
CONSTRAINT tarea_informes_tarea_id_fk FOREIGN KEY (tarea_id) REFERENCES
tareas(id)
);

CREATE INDEX idx_tarea_informes_tipo ON tarea_informes (tipo);
CREATE INDEX idx_tarea_informes_creado_en ON tarea_informes (creado_en);
CREATE INDEX idx_tarea_informes_usuario_id ON tarea_informes (usuario_id);
CREATE INDEX idx_tarea_informes_tarea_id ON tarea_informes (tarea_id);

```

5. Tabla: tarea_historial

Función de la tabla

La tabla tarea_historial registra los cambios realizados sobre las tareas. Su objetivo es mantener trazabilidad de los cambios de estado, comentarios y acciones ejecutadas por los usuarios.

Estructura de la tabla

- id: identificador único del registro.
- tarea_id: tarea afectada.
- usuario_id: usuario que realizó la acción.
- estado_anterior: estado previo de la tarea.
- estado_nuevo: nuevo estado de la tarea.

- comentario: comentario asociado al cambio.
- creado_en: fecha del registro.

Script de creación

```
CREATE TABLE tarea_historial (  
    id TEXT PRIMARY KEY,  
    tarea_id TEXT NOT NULL,  
    usuario_id TEXT NOT NULL,  
    estado_anterior TEXT,  
    estado_nuevo TEXT,  
    comentario TEXT,  
    creado_en TEXT NOT NULL,  
  
    CONSTRAINT tarea_historial_tarea_id_fk FOREIGN KEY (tarea_id) REFERENCES  
tareas(id),  
    CONSTRAINT tarea_historial_usuario_id_fk FOREIGN KEY (usuario_id) REFERENCES  
usuarios(id)  
);
```

```
CREATE INDEX idx_tarea_historial_creado_en ON tarea_historial (creado_en);  
CREATE INDEX idx_tarea_historial_usuario_id ON tarea_historial (usuario_id);  
CREATE INDEX idx_tarea_historial_tarea_id ON tarea_historial (tarea_id);
```

6. Tabla: tarea_asignaciones

Función de la tabla

La tabla tarea_asignaciones gestiona la relación entre tareas y usuarios asignados. Permite registrar roles, estados de participación y fechas clave del proceso, como selección, inicio, cancelación y finalización.

Estructura de la tabla

- id: identificador único de la asignación.
- tarea_id: tarea asignada.
- usuario_id: usuario asignado.
- rol: rol del usuario dentro de la tarea.
- estado: estado de la asignación.
- creado_en: fecha de creación.
- iniciado_en: fecha de inicio.
- completado_en: fecha de finalización.
- seleccionado_en: fecha de selección.
- cancelado_en: fecha de cancelación.

Script de creación

```
CREATE TABLE tarea_asignaciones (  
  id TEXT PRIMARY KEY,  
  tarea_id TEXT NOT NULL,  
  usuario_id TEXT NOT NULL,  
  rol TEXT NOT NULL DEFAULT 'miembro',  
  estado TEXT NOT NULL DEFAULT 'activo',  
  creado_en TEXT NOT NULL,  
  iniciado_en TEXT,  
  completado_en TEXT,  
  seleccionado_en TEXT,  
  cancelado_en TEXT,
```

```
CONSTRAINT tarea_asignaciones_rol_check CHECK (rol IN ('miembro', 'admin', 'owner')),  
CONSTRAINT tarea_asignaciones_estado_check CHECK (estado IN ('activo', 'cancelado',  
'completado', 'pausado')),  
CONSTRAINT tarea_asignaciones_tarea_id_fk FOREIGN KEY (tarea_id) REFERENCES  
tareas(id),  
CONSTRAINT tarea_asignaciones_usuario_id_fk FOREIGN KEY (usuario_id)  
REFERENCES usuarios(id)  
);
```

```
CREATE INDEX idx_ta_tarea_id ON tarea_asignaciones (tarea_id);  
CREATE INDEX idx_ta_usuario_id ON tarea_asignaciones (usuario_id);  
CREATE INDEX idx_ta_tarea_estado ON tarea_asignaciones (tarea_id, estado);  
CREATE UNIQUE INDEX idx_unique_activo ON tarea_asignaciones (tarea_id, usuario_id);
```

7. Tabla: supervisor_usuarios

Función de la tabla

La tabla supervisor_usuarios registra la relación entre supervisores y usuarios supervisados. Su finalidad es determinar qué colaborador está bajo seguimiento de qué supervisor dentro del módulo de jornada laboral.

Estructura de la tabla

- id: identificador único.
- supervisor_id: usuario que supervisa.
- usuario_id: usuario supervisado.
- fecha_inicio: fecha de inicio de supervisión.
- fecha_fin: fecha de finalización.
- creado_en: fecha de creación.

- actualizado_en: fecha de actualización.

Script de creación

```
CREATE TABLE supervisor_usuarios (  
    id TEXT PRIMARY KEY,  
    supervisor_id TEXT NOT NULL,  
    usuario_id TEXT NOT NULL,  
    fecha_inicio TEXT,  
    fecha_fin TEXT,  
    creado_en TEXT,  
    actualizado_en TEXT  
);
```

8. Tabla: registro_jornada

Función de la tabla

La tabla registro_jornada almacena el control diario de entrada, salida, estado y tiempo trabajado por cada usuario. Permite llevar un registro formal de la jornada laboral supervisada.

Estructura de la tabla

- id: identificador único del registro.
- usuario_id: usuario al que pertenece la jornada.
- supervisor_id: supervisor responsable.
- fecha: fecha del registro.
- hora_entrada: hora de ingreso.
- hora_salida: hora de salida.
- minutos_trabajados: total de minutos calculados.

- estado: estado de la jornada.
- motivo: motivo asociado, en caso de ausencia o justificación.
- creado_en: fecha de creación.
- actualizado_en: fecha de actualización.

Script de creación

```
CREATE TABLE registro_jornada (
  id TEXT PRIMARY KEY,
  usuario_id TEXT NOT NULL,
  supervisor_id TEXT NOT NULL,
  fecha NUMERIC NOT NULL,
  hora_entrada TEXT,
  hora_salida TEXT,
  minutos_trabajados INTEGER DEFAULT 0,
  estado TEXT NOT NULL,
  motivo TEXT,
  creado_en TEXT DEFAULT CURRENT_TIMESTAMP,
  actualizado_en TEXT
);
```

9. Tabla: registro_horas

Función de la tabla

La tabla registro_horas registra el tiempo dedicado por un usuario a una tarea específica. Permite almacenar la hora de inicio, pausa, detención, total acumulado y estado del cronómetro o seguimiento de trabajo.

Estructura de la tabla

- id: identificador único.
- tarea_id: tarea relacionada.
- usuario_id: usuario que registra el tiempo.
- iniciado_en: fecha y hora de inicio.
- pausado_en: fecha y hora de pausa.
- detenido_en: fecha y hora de finalización.
- total_segundos: total de tiempo acumulado.
- estado: estado actual del registro.
- creado_en: fecha de creación.

Script de creación

```
CREATE TABLE registro_horas (  
    id TEXT PRIMARY KEY,  
    tarea_id TEXT NOT NULL,  
    usuario_id TEXT NOT NULL,  
    iniciado_en TEXT,  
    pausado_en TEXT,  
    detenido_en TEXT,  
    total_segundos INTEGER DEFAULT 0,  
    estado TEXT NOT NULL,  
    creado_en TEXT NOT NULL,  
  
    CONSTRAINT fk_registro_horas_usuario_id_usuarios_id_fk FOREIGN KEY (usuario_id)  
REFERENCES usuarios(id),
```

```
CONSTRAINT fk_registro_horas_tarea_id_tareas_id_fk FOREIGN KEY (tarea_id)
REFERENCES tareas(id)
);
```

```
CREATE INDEX idx_registro_horas_usuario_id ON registro_horas (usuario_id);
CREATE INDEX idx_registro_horas_tarea_id ON registro_horas (tarea_id);
```

10. Tabla: proyecto_usuarios

Función de la tabla

La tabla proyecto_usuarios representa la relación entre usuarios y proyectos. Permite saber qué usuarios pertenecen a cada proyecto, cuál es su rol dentro del mismo y por qué medio se unieron.

Estructura de la tabla

- proyecto_id: proyecto asociado.
- usuario_id: usuario asociado.
- rol_en_proyecto: rol del usuario dentro del proyecto.
- fecha_union: fecha de incorporación.
- tipo_union: forma en que el usuario se integró al proyecto.

Script de creación

```
CREATE TABLE proyecto_usuarios (
    proyecto_id INTEGER NOT NULL,
    usuario_id TEXT NOT NULL,
    rol_en_proyecto TEXT NOT NULL,
    fecha_union TEXT,
    tipo_union TEXT,
```

```
CONSTRAINT proyecto_usuarios_rol_check CHECK (rol_en_proyecto IN ('owner', 'admin', 'miembro'))),
```

```
CONSTRAINT proyecto_usuarios_tipo_union_check CHECK (tipo_union IS NULL OR tipo_union IN ('owner', 'manual', 'publico', 'solicitud')),
```

```
CONSTRAINT proyecto_usuarios_usuario_id_fk FOREIGN KEY (usuario_id) REFERENCES usuarios(id),
```

```
CONSTRAINT proyecto_usuarios_proyecto_id_fk FOREIGN KEY (proyecto_id) REFERENCES proyectos(id),
```

```
CONSTRAINT proyecto_usuarios_unique UNIQUE (proyecto_id, usuario_id) );
```

```
CREATE INDEX idx_proyecto_usuarios_usuario_id ON proyecto_usuarios (usuario_id);
```

```
CREATE INDEX idx_proyecto_usuarios_proyecto_id ON proyecto_usuarios (proyecto_id);
```

11. Tabla: proyecto_solicitudes

Función de la tabla

La tabla proyecto_solicitudes almacena las solicitudes realizadas por usuarios para ingresar a proyectos que requieren aprobación. Permite controlar el estado de cada solicitud y su mensaje asociado.

Estructura de la tabla

- id: identificador único.
- proyecto_id: proyecto al que se solicita ingreso.
- usuario_id: usuario solicitante.
- estado: estado actual de la solicitud.
- mensaje: mensaje opcional del solicitante.
- creado_en: fecha de creación.

- actualizado_en: fecha de actualización.

Script de creación

```
CREATE TABLE proyecto_solicitudes (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    proyecto_id INTEGER NOT NULL,  
    usuario_id TEXT NOT NULL,  
    estado TEXT NOT NULL DEFAULT 'pendiente',  
    mensaje TEXT,  
    creado_en NUMERIC DEFAULT CURRENT_TIMESTAMP,  
    actualizado_en NUMERIC DEFAULT CURRENT_TIMESTAMP,  
  
    CONSTRAINT proyecto_solicitudes_estado_check CHECK (estado IN ('pendiente',  
'aprobada', 'rechazada')),  
    CONSTRAINT proyecto_solicitudes_proyecto_id_fk FOREIGN KEY (proyecto_id)  
REFERENCES proyectos(id),  
    CONSTRAINT proyecto_solicitudes_usuario_id_fk FOREIGN KEY (usuario_id)  
REFERENCES usuarios(id),  
    CONSTRAINT proyecto_solicitudes_unique UNIQUE (proyecto_id, usuario_id)  
);  
  
CREATE INDEX idx_proyecto_solicitudes_estado ON proyecto_solicitudes (estado);  
CREATE INDEX idx_proyecto_solicitudes_usuario_id ON proyecto_solicitudes (usuario_id);  
CREATE INDEX idx_proyecto_solicitudes_proyecto_id ON proyecto_solicitudes (proyecto_id);
```

12. Tabla: sesiones_trabajo

Función de la tabla

La tabla sesiones_trabajo registra las sesiones activas de los usuarios dentro del sistema. Permite controlar el inicio y fin de sesión, así como llevar un seguimiento del tiempo de conexión y actividad del usuario.

Estructura de la tabla

- id: identificador único de la sesión
- usuario_id: usuario que inicia la sesión
- inicio: fecha y hora de inicio de la sesión
- fin: fecha y hora de finalización
- actualizado_en: última actualización del registro

Script de creación

```
CREATE TABLE sesiones_trabajo (  
    id TEXT PRIMARY KEY,  
    usuario_id TEXT NOT NULL,  
    inicio TEXT NOT NULL,  
    fin TEXT,  
    actualizado_en TEXT,  
  
    CONSTRAINT fk_sesiones_trabajo_usuario_id_usuarios_id_fk  
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id)  
);
```

Configuración del Entorno (.env.local)

Función

El archivo .env.local contiene las variables de entorno necesarias para la correcta configuración y funcionamiento del sistema. Estas variables permiten establecer la conexión con la base de datos, así como la integración con servicios externos como el envío de correos electrónicos.

Por motivos de seguridad, este archivo no se incluye en el repositorio público y debe ser configurado manualmente por el usuario.

Script:

```
TURSO_DATABASE_URL=tu_url_de_base_de_datos  
TURSO_AUTH_TOKEN=tu_token_de_autenticacion  
RESEND_API_KEY=tu_api_key  
FROM_EMAIL=nombre correo@dominio.com
```

Descripción de variables

- TURSO_DATABASE_URL

Define la URL de conexión a la base de datos en Turso.

- TURSO_AUTH_TOKEN

Token de autenticación necesario para acceder a la base de datos.

- RESEND_API_KEY

Clave de acceso al servicio de envío de correos electrónicos proporcionado por Resend.

- FROM_EMAIL

Correo remitente utilizado para el envío de notificaciones del sistema.

Conexión a la base de datos

(lib/database.ts)

Función del archivo

El archivo database.ts se encarga de establecer la conexión entre la aplicación y la base de datos utilizando el cliente de Turso.

Permite ejecutar consultas SQL desde los distintos módulos del sistema de forma centralizada.

Código

```
import { createClient } from '@libsql/client';
console.log('URL:', process.env.TURSO_DATABASE_URL);
console.log('TOKEN:', process.env.TURSO_AUTH_TOKEN ? 'OK' : 'MISSING');
export const db = createClient({
  url: process.env.TURSO_DATABASE_URL!,
  authToken: process.env.TURSO_AUTH_TOKEN!,
});
```

Explicación técnica

createClient: crea una instancia de conexión con la base de datos.

TURSO_DATABASE_URL: URL de conexión.

TURSO_AUTH_TOKEN: token de autenticación.

Se exporta db para reutilizarlo en toda la aplicación.

Servicio de envío de correos

(lib/email.ts)

Función del archivo

El archivo email.ts configura el servicio de envío de correos electrónicos mediante la API de Resend.

Este servicio se utiliza para funcionalidades como:

- Recuperación de contraseña
- Notificaciones del sistema

Código

```
import { Resend } from 'resend';  
export const resend = new Resend(process.env.RESEND_API_KEY);
```

Explicación técnica

Se crea una instancia de Resend usando la API Key.

Permite enviar correos desde cualquier parte del sistema.

Gestión de países

(lib/countries.ts)

Función del archivo

El archivo countries.ts contiene una lista de países con sus respectivos códigos telefónicos, banderas y nombres.

Se utiliza principalmente en formularios de registro y validación de usuarios.

Código

```
export interface pais {
  code: string;
  pais: string;
  flag: string;
  name: string;
}

export const countries: pais[] = [
  // América del Norte
  { code: '+1', pais: 'US', flag: 'us', name: 'Estados Unidos' },
  { code: '+1', pais: 'CA', flag: 'CA', name: 'Canadá' },
  { code: '+52', pais: 'MX', flag: 'MX', name: 'México' },

  // América Central
  { code: '+501', pais: 'BZ', flag: 'BZ', name: 'Belize' },
  { code: '+506', pais: 'CR', flag: 'CR', name: 'Costa Rica' },
  { code: '+503', pais: 'SV', flag: 'SV', name: 'El Salvador' },
  { code: '+502', pais: 'GT', flag: 'GT', name: 'Guatemala' },
  { code: '+504', pais: 'HN', flag: 'HN', name: 'Honduras' },
  { code: '+505', pais: 'NI', flag: 'NI', name: 'Nicaragua' },
  { code: '+507', pais: 'PA', flag: 'PA', name: 'Panamá' },

  // América del Sur
  { code: '+54', pais: 'AR', flag: 'AR', name: 'Argentina' },
  { code: '+591', pais: 'BO', flag: 'BO', name: 'Bolivia' },
  { code: '+55', pais: 'BR', flag: 'BR', name: 'Brasil' },
```

```

{ code: '+56', pais: 'CL', flag: 'CL', name: 'Chile' },
{ code: '+57', pais: 'CO', flag: 'CO', name: 'Colombia' },
{ code: '+593', pais: 'EC', flag: 'EC', name: 'Ecuador' },
{ code: '+592', pais: 'GY', flag: 'GY', name: 'Guyana' },
{ code: '+595', pais: 'PY', flag: 'PY', name: 'Paraguay' },
{ code: '+51', pais: 'PE', flag: 'PE', name: 'Perú' },
{ code: '+597', pais: 'SR', flag: 'SR', name: 'Surinam' },
{ code: '+598', pais: 'UY', flag: 'UY', name: 'Uruguay' },
{ code: '+58', pais: 'VE', flag: 'VE', name: 'Venezuela' },

// Caribe
{ code: '+1', pais: 'PR', flag: 'PR', name: 'Puerto Rico' },
{ code: '+1', pais: 'DO', flag: 'DO', name: 'República Dominicana' },
{ code: '+53', pais: 'CU', flag: 'CU', name: 'Cuba' },

// Europa (principales)
{ code: '+34', pais: 'ES', flag: 'ES', name: 'España' },
{ code: '+44', pais: 'GB', flag: 'GB', name: 'Reino Unido' },
{ code: '+33', pais: 'FR', flag: 'FR', name: 'Francia' },
{ code: '+49', pais: 'DE', flag: 'DE', name: 'Alemania' },
{ code: '+39', pais: 'IT', flag: 'IT', name: 'Italia' },
{ code: '+351', pais: 'PT', flag: 'PT', name: 'Portugal' },
];

// Buscar país por ISO (recomendado)
export const findpaisByISO = (iso: string): pais | undefined => {

```

```

    return countries.find(c => c.pais === iso);
  };

  // Buscar país por ISO + código (validación fuerte)
  export const isValidpais = (iso: string, code: string): boolean => {
    return countries.some(c => c.pais === iso && c.code === code);
  };

  // Función helper para obtener el país por defecto (Costa Rica)
  export const getDefaultpais = (): pais => {
    return countries.find(c => c.pais === 'CR') || countries[0];
  };

```

Funcionalidades principales:

- Lista de países

Contiene un arreglo con información de distintos países.

- Buscar país por ISO

```

export const findpaisByISO = (iso: string): pais | undefined => {

  return countries.find(c => c.pais === iso);

};

```

- Validar país

```

export const isValidpais = (iso: string, code: string): boolean => {

  return countries.some(c => c.pais === iso && c.code === code);

};

```

- Obtener país por defecto

```
export const getDefaultpais = (): pais => {  
  
  return countries.find(c => c.pais === 'CR') || countries[0];  
  
};
```

Explicación técnica

Este módulo permite:

- Validar datos de entrada del usuario
- Estandarizar países y códigos telefónicos
- Mejorar la experiencia en formularios

Autenticación

(lib/auth.ts)

Función del archivo

El archivo auth.ts gestiona la autenticación y autorización de usuarios dentro del sistema.

Permite:

- Obtener el usuario desde la sesión
- Validar si está autenticado
- Controlar accesos según roles

Código

```
import { NextRequest } from "next/server";
import { db } from "@/lib/database";

/**
 * =====
 * 🗝️ Helper básico (mantener por compatibilidad)
 * =====
 * Obtiene el userId desde la cookie de sesión
 */
export function getUserIdFromRequest(req: NextRequest): string | null {
  return req.cookies.get("session_user_id")?.value ?? null;
}

/**
 * =====
 * 🧑 Tipo de usuario autenticado
 * =====
 */
export type AuthUser = {
  id: string;
  nombre: string;
  apellido: string;
  email: string;
  rol: string;
};

/**
```

* =====

*  Helper principal recomendado

* =====

* - Valida sesión

* - Verifica que el usuario esté activo

* - Devuelve el usuario listo para usar

*/

```
export async function getAuthenticatedUser(
  req: NextRequest
): Promise<AuthUser | null> {
  try {
    const userId = getUserIdFromRequest(req);

    if (!userId) return null;

    const result = await db.execute({
      sql: `
        SELECT id, nombre, apellido, email, rol, activo
        FROM usuarios
        WHERE id = ?
        LIMIT 1
      `,
      args: [userId],
    });

    const user = result.rows[0] as any;

    //  Validación crítica
```

```

    if (!user || Number(user.activo ?? 0) !== 1) {
      return null;
    }

    return {
      id: String(user.id),
      nombre: String(user.nombre ?? ""),
      apellido: String(user.apellido ?? ""),
      email: String(user.email ?? ""),
      rol: String(user.rol ?? "colaborador"),
    };
  } catch (error) {
    console.error("getAuthenticatedUser error:", error);
    return null;
  }
}

/**
 * =====
 * 🛡️ Helper para proteger rutas (opcional pero PRO)
 * =====
 * Lanza error si no está autenticado
 */
export async function requireUser(req: NextRequest): Promise<AuthUser> {
  const user = await getAuthenticatedUser(req);
  if (!user) {
    throw new Error("UNAUTHORIZED");
  }
}

```

```

    return user;
}

/**
 * =====
 * Helper de roles (muy útil para tu sistema tipo Jira)
 * =====
 */
export function isAdmin(user: AuthUser): boolean {
    return user.rol === "admin" || user.rol === "jefe";
}

export function isJefe(user: AuthUser): boolean {
    return user.rol === "jefe";
}

```

Funciones principales

- Obtener ID desde la sesión

```

export function getUserIdFromRequest(req: NextRequest): string | null {

    return req.cookies.get("session_user_id")?.value ?? null;

}

```

Obtiene el ID del usuario desde cookies

- Obtener usuario autenticado

```

export async function getAuthenticatedUser(

```

req: NextRequest

): Promise<AuthUser | null>

Función principal:

- Verifica sesión
- Consulta la base de datos
- Valida que el usuario esté activo
- Proteger rutas

```
export async function requireUser(req: NextRequest): Promise<AuthUser>
```

Lanza error si el usuario no está autenticado

- Validación de roles

```
export function isAdmin(user: AuthUser): boolean {
```

```
  return user.rol === "admin" || user.rol === "jefe";
```

```
}
```

```
export function isJefe(user: AuthUser): boolean {
```

```
  return user.rol === "jefe";
```

```
}
```

Permiten controlar permisos dentro del sistema

Explicación técnica

Este módulo centraliza la lógica de autenticación, asegurando que:

- Solo usuarios válidos accedan al sistema
- Se respeten roles y permisos
- Se mantenga la seguridad de las rutas

1. Endpoints de autenticación

1.1.Inicio de sesión

// Endpoint: /api/auth/login

// Método: POST

export async function POST(req: NextRequest)

- Permite autenticar al usuario mediante correo electrónico y contraseña, validando sus credenciales y generando una sesión activa en el sistema.

1.2.Cierre de sesión

// Endpoint: /api/auth/logout

// Método: POST

export async function POST(req: NextRequest)

Finaliza la sesión activa del usuario, actualizando el registro en la base de datos y eliminando las cookies de autenticación.

1.3.Usuario autenticado

// Endpoint: /api/auth/me

// Método: GET

export async function GET(req: NextRequest)

- Verifica si el usuario se encuentra autenticado, retornando sus datos si la sesión es válida.

1.4.Recuperación de contraseña

// Endpoint: /api/auth/forgot-password

// Método: POST

export async function POST(req: Request)

- Genera un código temporal de recuperación de contraseña, lo almacena en la base de datos con un tiempo de expiración y envía dicho código al correo electrónico del usuario.

1.5.Verificación de código

// Endpoint: /api/auth/verify-code

// Método: POST

export async function POST(req: Request)

- Valida que el código de recuperación ingresado por el usuario sea correcto, tenga el formato esperado y no haya expirado.

1.6.Restablecimiento de contraseña

// Endpoint: /api/auth/reset-password

// Método: POST

export async function POST(req: Request)

- Permite actualizar la contraseña del usuario validando previamente el correo, el código de recuperación y las reglas de seguridad de la nueva contraseña. Además, envía un correo de confirmación tras el cambio.

1.7.Registro de usuario

// Endpoint: /api/auth/register

// Método: POST

export async function POST(req: Request)

- Permite registrar un nuevo usuario en el sistema, validando nombre, apellido, correo, contraseña, país y teléfono. Además, cifra la contraseña, verifica que el correo no exista previamente y crea la cuenta con estado pendiente de activación.

2. Endpoints del módulo de jornada laboral

2.1.Consulta de registros de jornada

// Endpoint: /api/jornada

// Método: GET

export async function GET(req: NextRequest)

- Obtiene los registros de jornada de una fecha específica para el supervisor autenticado, mostrando la información de los usuarios asignados y sus datos de entrada, salida, estado y minutos trabajados.

2.2.Registro de jornada

// Endpoint: /api/jornada

// Método: POST

export async function POST(req: NextRequest)

- Permite crear o actualizar un registro de jornada laboral para un usuario asignado al supervisor autenticado, validando fecha, estado, horas de entrada y salida, motivo y cálculo de minutos trabajados.

2.3.Consulta de usuarios de jornada

// Endpoint: /api/jornada/usuarios

// Método: GET

export async function GET(req: NextRequest)

- Obtiene la lista de usuarios disponibles o asignados al supervisor dentro del módulo de jornada, según el modo de consulta indicado.

2.4.Asignación de usuarios a jornada

// Endpoint: /api/jornada/usuarios

// Método: POST

export async function POST(req: NextRequest)

- Permite asignar o actualizar los usuarios supervisados por el administrador o jefe autenticado, controlando fechas de inicio y cierre de asignaciones.

2.5.Actualización de un registro de jornada

// Endpoint: /api/jornada/[id]

// Método: PUT

```
export async function PUT(
```

```
  req: NextRequest,
```

```
  { params }: { params: Promise<{ id: string }> }
```

```
)
```

- Actualiza un registro de jornada específico, validando el estado, las horas registradas y el motivo correspondiente, siempre que pertenezca al supervisor autenticado.

2.6.Eliminación de un registro de jornada

// Endpoint: /api/jornada/[id]

// Método: DELETE

```
export async function DELETE(
```

```
  req: NextRequest,
```

```
  { params }: { params: Promise<{ id: string }> }
```

```
)
```

- Elimina un registro de jornada específico, siempre que exista y esté asociado al supervisor autenticado.

3. Endpoints del módulo de proyectos

3.1.Consulta general de proyectos

// Endpoint: /api/proyectos

// Método: GET

```
export async function GET(req: NextRequest)
```

- Obtiene los proyectos visibles para el usuario autenticado, permitiendo filtrar proyectos creados por él, proyectos donde es miembro o todos los proyectos a los que tiene acceso.

3.2.Creación de proyecto

// Endpoint: /api/proyectos

// Método: POST

```
export async function POST(req: NextRequest)
```

- Permite crear un nuevo proyecto, validando nombre, prioridad, visibilidad, modo de acceso, fechas y miembros asociados. Además, registra al creador como propietario del proyecto y agrega los miembros indicados.

3.3.Consulta de proyecto específico

// Endpoint: /api/proyectos/[id]

// Método: GET

```
export async function GET(
```

```
  req: NextRequest,
```

```
{ params }: { params: Promise<{ id: string }> }
```

)

- Obtiene la información detallada de un proyecto específico, validando autenticación, membresía, permisos de acceso y capacidad de edición o gestión de tareas según el rol del usuario dentro del proyecto.

4. Endpoints avanzados del módulo de proyectos

4.1.Consulta de configuración del proyecto

```
// Endpoint: /api/proyectos/[id]/configuracion
```

```
// Método: GET
```

```
export async function GET(req: NextRequest, { params }: ParamsContext)
```

- Obtiene la configuración completa de un proyecto específico, incluyendo su información general, permisos de edición, permisos de gestión de tareas, rol del usuario autenticado y catálogos de valores válidos para configuración.

4.2.Actualización de configuración del proyecto

```
// Endpoint: /api/proyectos/[id]/configuracion
```

```
// Método: PATCH
```

```
export async function PATCH(req: NextRequest, { params }: ParamsContext)
```

- Permite actualizar la configuración de un proyecto, validando permisos del usuario, estado, visibilidad, modo de acceso, prioridad, fechas y reglas de edición o gestión de tareas.

4.3.Eliminación de proyecto

// Endpoint: /api/proyectos/[id]/configuracion

// Método: DELETE

```
export async function DELETE(req: NextRequest, { params }: ParamsContext)
```

- Elimina un proyecto de forma controlada, borrando previamente las relaciones asociadas a sus tareas y dependencias, siempre que el usuario autenticado sea el propietario del proyecto.

4.4.Consulta de miembros del proyecto

// Endpoint: /api/proyectos/[id]/miembros

// Método: GET

```
export async function GET(
```

```
  request: NextRequest,
```

```
  context: { params: Params | Promise<Params> }  
)
```

- Obtiene la lista de miembros de un proyecto, incluyendo datos personales básicos, rol dentro del proyecto, fecha de unión y cantidad de tareas activas asignadas.

4.5.Agregar miembro al proyecto

// Endpoint: /api/proyectos/[id]/miembros

// Método: POST

```
export async function POST(
```

```
  request: NextRequest,
```

```
  context: { params: Params | Promise<Params> }  
)
```

- Permite agregar un nuevo miembro a un proyecto, validando permisos del usuario autenticado, existencia del usuario y evitando duplicados dentro del proyecto.

4.6.Actualizar rol de miembro

// Endpoint: /api/proyectos/[id]/miembros

// Método: PATCH

```
export async function PATCH(
```

```
  request: NextRequest,
```

```
  context: { params: Params | Promise<Params> }  
)
```

- Permite modificar el rol de un miembro dentro del proyecto, respetando las reglas de permisos y evitando cambios sobre el propietario del proyecto.

4.7.Eliminar miembro del proyecto

// Endpoint: /api/proyectos/[id]/miembros

// Método: DELETE

export async function DELETE(

request: NextRequest,

context: { params: Params | Promise<Params> }
)

- Elimina un miembro de un proyecto y, opcionalmente, libera sus tareas activas asociadas dentro del mismo proyecto.

4.8.Vista previa del proyecto

// Endpoint: /api/proyectos/[id]/preview

// Método: GET

export async function GET(req: NextRequest, { params }: RouteContext)

- Obtiene una vista previa del proyecto, mostrando información general, miembros, tareas, estadísticas y permisos de acceso según el modo de visibilidad y la relación del usuario con el proyecto.

5. Endpoints de acceso y solicitudes de proyectos

5.1.Unirse a un proyecto público

// Endpoint: /api/proyectos/[id]/unirse

// Método: POST

export async function POST(

```
req: NextRequest,  
  
  { params }: { params: Promise<{ id: string }> }  
  
)
```

- Permite que un usuario autenticado se una directamente a un proyecto público, validando que el proyecto exista, que permita unión directa y que el usuario no sea ya miembro o creador.

5.2.Crear solicitud de acceso a proyecto

```
// Endpoint: /api/proyectos/[id]/solicitudes
```

```
// Método: POST
```

```
export async function POST(  
  
  req: NextRequest,  
  
  { params }: { params: Promise<{ id: string }> }  
  
)
```

- Permite crear una solicitud de acceso para un proyecto configurado en modo de solicitud, validando que el usuario no sea ya miembro ni creador del proyecto.

5.3.Consultar solicitudes pendientes

```
// Endpoint: /api/proyectos/[id]/solicitudes
```

```
// Método: GET
```

```
export async function GET(  
  
)
```

```
req: NextRequest,  
  
  { params }: { params: Promise<{ id: string }> }  
  
)
```

- Obtiene la lista de solicitudes pendientes de acceso a un proyecto, mostrando información básica del solicitante. Solo puede ser consultado por el creador del proyecto.

5.4.Procesar solicitud de acceso

// Endpoint: /api/proyectos/[id]/solicitudes/[solicitudId]

// Método: POST

```
export async function POST(  
  
  req: NextRequest,  
  
  { params }: { params: Promise<{ id: string; solicitudId: string }> }  
  
)
```

- Permite aprobar o rechazar una solicitud de acceso a proyecto. Si la solicitud es aprobada, el usuario solicitante es agregado como miembro del proyecto.

6. Endpoints del módulo de tareas

6.1.Consulta de tareas del proyecto

// Endpoint: /api/proyectos/[id]/tareas

// Método: GET

```
export async function GET(  
  

```

```
req: NextRequest,  
  
  { params }: { params: Promise<{ id: string }> }  
  
)
```

- Obtiene la lista de tareas asociadas a un proyecto, validando el acceso del usuario según el modo de visibilidad y su relación con el proyecto.

6.2.Creación de tarea

```
// Endpoint: /api/proyectos/[id]/tareas
```

```
// Método: POST
```

```
export async function POST(  
  
  req: NextRequest,  
  
  { params }: { params: Promise<{ id: string }> }  
  
)
```

- Permite crear una nueva tarea dentro de un proyecto, validando permisos de gestión, prioridad, tiempo estimado, participantes máximos y registrando el evento en el historial de tareas.

6.3.Aprobación de tarea

```
// Endpoint: /api/proyectos/[id]/tareas/[tareaId]/aprobar
```

```
// Método: POST
```

```
export async function POST(  
  

```

```
req: NextRequest,
```

```
{ params }: { params: Promise<{ id: string; tareaId: string }> }
```

```
)
```

- Permite aprobar una tarea que se encuentra en estado de revisión, cambiando su estado a completada, registrando la fecha de aprobación, el usuario aprobador y el historial del cambio.

6.4.Consulta de asignados de una tarea

```
// Endpoint: /api/proyectos/[id]/tareas/[tareaId]/asignados
```

```
// Método: GET
```

```
export async function GET(
```

```
req: NextRequest,
```

```
{ params }: { params: Promise<{ id: string; tareaId: string }> }
```

```
)
```

- Obtiene la lista de usuarios asignados activamente a una tarea, incluyendo información de selección, inicio, finalización y estado del cronómetro asociado.

6.5.Cancelación de selección de tarea

```
// Endpoint: /api/proyectos/[id]/tareas/[tareaId]/cancelar
```

```
// Método: POST
```

```
export async function POST(
```

```
req: NextRequest,  
  
context: { params: Params | Promise<Params> }  
  
)
```

- Permite cancelar la participación activa de un usuario en una tarea, finalizando su registro de tiempo si existe, actualizando su asignación y recalculando el estado general de la tarea.

6.6.Comenzar tarea (iniciar cronómetro)

// Endpoint: /api/proyectos/[id]/tareas/[tareaId]/comenzar

// Método: POST

```
export async function POST(  
  

```

```
req: NextRequest,  
  

```

```
{ params }: { params: Promise<{ id: string; tareaId: string }> }  
  
)
```

- Permite a un usuario iniciar una tarea previamente seleccionada, cambiando su estado a in-progress, registrando la hora de inicio y activando o creando el cronómetro de trabajo.

6.7.Envíar tarea a revisión (completar)

// Endpoint: /api/proyectos/[id]/tareas/[tareaId]/completar

// Método: POST

```
export async function POST(  
  

```

```
req: NextRequest,
```

```
{ params }: { params: Promise<{ id: string; tareaId: string }> }
```

```
)
```

- Permite marcar una tarea como finalizada por el usuario asignado, deteniendo el cronómetro y cambiando el estado a review para posterior aprobación.

6.8.Actualizar estado de tarea (control total)

```
// Endpoint: /api/proyectos/[id]/tareas/[tareaId]/estado
```

```
// Método: PATCH
```

```
export async function PATCH(
```

```
req: NextRequest,
```

```
context: { params: Params | Promise<Params> }
```

```
)
```

- Permite cambiar el estado de una tarea siguiendo reglas estrictas de flujo (todo → in-progress → review → completed) incluyendo acciones como rechazo o reapertura, con validación de permisos y registro en historial.

6.9.Informes de tarea (avance / final)

```
// Endpoint: /api/proyectos/[id]/tareas/[tareaId]/informes
```

```
// Métodos: GET, POST
```

- Permite gestionar los informes asociados a una tarea:

GET → Obtener todos los informes de la tarea

POST → Crear informe (avance o final)

- Reglas importantes:

Solo usuarios asignados pueden crear informes

No se pueden crear informes si la tarea está en:

- review
- completed

Al subir informe:

- Se limpia el rechazo anterior
- Se registra en historial

6.10. Rechazar tarea

// Endpoint: /api/proyectos/[id]/tareas/[tareaId]/rechazar

// Método: POST

- Permite devolver una tarea de review → in-progress con comentario obligatorio.

Reglas:

- Solo owner o admin
- Solo si la tarea está en review

Guarda comentario en:

ultimo_rechazo_comentario

tarea_historial

6.11. Seleccionar tarea

// Endpoint: /api/proyectos/[id]/tareas/[tareaId]/seleccionar

// Método: POST

- Permite que un usuario se asigne a una tarea.

Características:

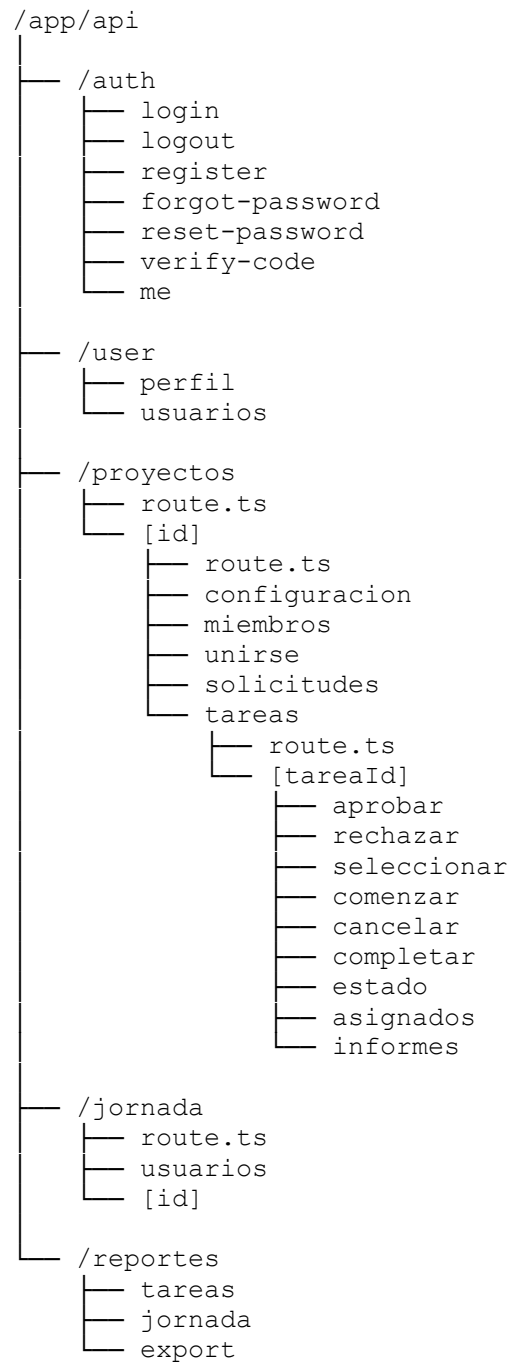
- Respeta max_participantes
- Evita duplicados
- Reactivas asignaciones anteriores
- Controla estados (activo, cancelado, etc.)

Estructura del Backend

El backend del sistema está implementado utilizando las API Routes de Next.js, organizadas bajo el directorio `/app/api`, siguiendo una estructura modular basada en funcionalidades.

Estructura general

Esquema backend



Explicacion esquema:

El backend sigue un enfoque basado en:

- API RESTful
- Arquitectura modular por dominio
- Separación de responsabilidades

Componentes principales

1. API Routes

- Manejan solicitudes HTTP (GET, POST, PATCH, DELETE)
- Contienen la lógica del negocio
- Validan permisos y roles

2. Capa de acceso a datos

- Implementada mediante el cliente de Turso
- Permite ejecutar consultas SQL directamente

3. Módulos auxiliares (/lib)

- Autenticación
- Conexión a base de datos
- Envío de correos
- Validaciones

Flujo del Backend

El flujo típico de una operación es el siguiente:

1. El cliente realiza una solicitud HTTP
2. La API valida autenticación y permisos
3. Se procesa la lógica de negocio
4. Se ejecuta una consulta a la base de datos
5. Se retorna una respuesta en formato JSON

Lógica del sistema de tareas

El sistema implementa un flujo controlado de estados para garantizar la integridad del proceso:

todo → in-progress → review → completed

Reglas principales:

1. Una tarea debe ser seleccionada antes de comenzar
2. Solo usuarios asignados pueden trabajar en la tarea
3. Para completar una tarea:
 - Debe estar en `in-progress`
 - Debe enviarse a `review`

4. Solo **admin o owner** pueden aprobar o rechazar
5. Si se rechaza:
 - Vuelve a `in-progress`
 - Se guarda comentario obligatorio

Control de tiempo (registro_horas)

El sistema incluye un cronómetro interno para cada tarea:

Estados del registro:

- activo → trabajando
- pausado → en pausa
- finalizado → terminado

Funcionalidad:

- Inicio automático al comenzar tarea
- Pausa y reanudación
- Cálculo de tiempo en segundos
- Al completar:
 - Se detiene el cronómetro
 - Se guarda el tiempo total

Seguridad del Backend

El sistema implementa múltiples capas de seguridad:

1. Autenticación

- Uso de cookies de sesión (session_user_id)
- Validación en cada endpoint

2. Autorización

Roles:

- admin
- jefe
- colaborador

Roles por proyecto:

- owner
- admin
- miembro

3. Validaciones

- Validación de parámetros
- Validación de estados
- Validación de transiciones de flujo

4. Protección de rutas

Uso del helper:

```
getAuthenticatedUser(req)
```

Manejo de errores

Todos los endpoints implementan manejo de errores estructurado:

- 400 → Parámetros inválidos
- 401 → No autenticado
- 403 → Sin permisos
- 404 → Recurso no encontrado
- 409 → Conflicto de estado
- 500 → Error interno

Buenas prácticas implementadas

- Uso de UUID para identificadores
- Separación de lógica por módulos
- Validación estricta de estados
- Registro de historial (auditoría)
- Uso de índices en base de datos
- Código escalable y mantenible

Conclusión técnica del backend

El backend del sistema está diseñado bajo principios modernos de desarrollo, asegurando:

- Escalabilidad
- Seguridad
- Mantenibilidad
- Trazabilidad de acciones

Esto permite que la aplicación funcione de manera robusta en entornos empresariales, garantizando el control de procesos, usuarios y actividades.